BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

LESSON 034 · C# Login Project · Complete and Test

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. In a seven-bucket hash table, a collision occurs at bucket 6. Which bucket is checked next by linear probing, and why?

Q2. Predict the output of Console.WriteLine((age >= 16) && hasPermission); when age = 17 and hasPermission = false. Explain.

Q3. In a binary search tree, a new key is greater than the root but smaller than the root’s right child. Where will it go if that right child has no left child?

Q4. A print server must process jobs in the order submitted. Choose a suitable data structure and explain why a stack would be unsuitable.

Do Now Recall previous learning | **I Do** Read new knowledge | **We Do** Practise with guidance | **You Do** Work independently | **Review & Improve** Check understanding

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

validation — Confirm data meets input rules.

duplicate — A repeated key that must not be inserted twice.

boundary test — Check a value at a limit, such as a full table.

erroneous test — Use invalid inputs to check handling.

linear probing — Search sequential buckets for an available place.

authentication — Check whether supplied credentials should grant access.

synoptic — Combining knowledge from multiple specification areas.

New learning

Your goal: a reliable demonstration system — Students will join registration and lookup so the program recognises existing usernames, refuses duplicates and performs login checks.

Validation before storage — Reject blank usernames, duplicates and registrations when all buckets are full. Show a meaningful reason rather than crashing.

Separate locating an account from checking a password — First use the hash table to find the username. Then compare an invented classroom demo password. These are separate operations.

Design the test evidence — Test normal, erroneous and boundary scenarios: new user, unknown user, duplicate username, collision, table full and maximum attempts.

Security boundary — The simple hash function here is unsuitable for real authentication. Production systems need established salted password-hashing libraries and secure storage.

Synoptic retrieval — Exchange MCQs, answer independently, discuss distractors, then correct any technical errors. Cover the full OCR data-structures checklist.

Worked example

Model validation before registration

1. Check blank input — CODE:
Console.Write("New username: ");
string username = (Console.ReadLine() ?? "").Trim();
if (username.Length == 0)
{
Console.WriteLine("Username required");

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

False. Two keys can hash to the same index. Collision resolution, such as linear probing, prevents one from simply overwriting the other.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. Bucket 0: (6 + 1) MOD 7 = 0, so probing wraps around.

Q2. False. The age test is true but hasPermission is false, so the AND expression is false.

Q3. It becomes the left child of the root’s right child.

Q4. Use a queue: FIFO serves the earliest job first. A stack uses LIFO and would process the most recently added job first.

You Do pages · Independent questions

Q1. CODING 1 · COMPLETE REGISTER
Register: reject empty usernames; look up existing names first; reject duplicates; check for free buckets; insert using the same hash and linear-probing routine from Lesson 033. A full table must exit cleanly.

Q2. CODING 2 · COMPLETE LOGIN
First FindUser(name). If no match, access denied. If found, compare the supplied demonstration password against the associated DEMO value for that user. This is a classroom simulation: plaintext password comparisons and the simple table hash are not secure production practices.

Q3. CODING 3 · TEST TABLE
Sample tests: registered username+correct demo password → accept; correct username+wrong demo password → deny; unknown name → deny; duplicate name → refuse; 'ab'+'ba' → store and retrieve both; all 7 buckets occupied → reject extra registration without hanging. Add actual outcomes.

Q4. SYNOPTIC MCQ ASSESSMENT
MCQ marking: cover exactly the structures in the lesson specification. Array indexing/dimensions; records vs tuples; list resizing; linked-list pointers; directed graph edges; LIFO/FIFO; tree hierarchy and BST ordering; hash keys and collisions. Check each distractor and correct misconceptions.

Exit Ticket · Model answer

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

```
}
```

2. Check whether the username exists — Use the lookup function from Lesson 033. Do not register duplicates, including usernames moved by probing.

3. Enforce a size limit — Track how many table buckets are occupied. If all seven are full, refuse a new account gracefully.

We Do · Practise with guidance

Protect against duplicates

Which checks must happen before insertion?

Explain your solution first

```
if (username.Length == 0)
    Console.WriteLine("Username required");
else if (FindUser(username) >= 0)
    Console.WriteLine("Username exists");
else
    Console.WriteLine("Ready to register");
```

Test a collision

How do we prove the collision handler works?

Register 'ab' and 'ba'. Check both receive distinct buckets and FindUser locates both.

Build a synoptic MCQ

Which structure supports FIFO processing? Provide three plausible alternatives and justify the correct response.

Queue is correct. A stack is LIFO; a binary search tree organises ordered keys; a hash table is optimised for key-based lookup.

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

CODING 1 · COMPLETE REGISTER
Extend the Lesson 033 table with registration, blank-name checks, duplicate-name rejection and full-table handling. Keep existing hash and lookup functions.

Question 2

CODING 2 · COMPLETE LOGIN
Locate a registered username using the hash table. Then check an invented demonstration password associated with that account. Reject unknown usernames. Never enter real credentials.

Question 3

CODING 3 · TEST TABLE
Evidence at least six tests: successful login, wrong demo password, unknown username, duplicate, collision, full-table boundary. Record input, expected, actual and pass/fail.

Question 4

SYNOPTIC MCQ ASSESSMENT
Exchange student-written MCQs and answer independently. Cover arrays up to 3D, records, lists, tuples, linked lists, directed/undirected graphs, stacks, queues, trees, BSTs and hash tables. Identify and correct ambiguous distractors.

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

Exit Ticket · Answer before checking

Two usernames producing the same hash index must always overwrite one another.